Function Composition

Create a function named `compose` that:

- Accepts any number of functions as arguments
- Returns a new function representing their composition
- Passes arguments to the first function
- Passes each function's output to the next function
- Returns the final output

The provided functions are:

- `add(*args)`: Returns the sum of all arguments
- `square(a)`: Returns the square of a number
- `splitter(a)`: Divides a number by 2 and returns `[(a + 1) // 2, (a + 1) // 2]`
- `my_max(a)`: Returns the maximum value in a number or list
- `my_min(a)`: Returns the minimum value in a number or list

Note: Only the first function handles multiple arguments; subsequent functions take a single argument.

Example

`functionsList = [add, splitter]`

The function call `compose(functionsList[1], functionsList[2])` should return a function, call it `composedFunctions`. If `argumentsList = [2, 3]`, `composedFunctions(2, 3)`, should return `[3, 3]`.

- `add(2, 3) -> 5`
- `splitter(5) -> [3, 3]`

Function Description

Complete the function `compose` in the editor with the following parameters:

- `*functionsList`: function arguments

Returns

- `function`: the composition of `functionsList`

Constraints

- 1 <= n <= 15
- `functionsList[0]` = add
- `functionsList[i]` = {square, splitter, my_max, my_min}, where i != 0
- 1 <= q <= 10, length of `argumentsList`
- 1 <= `argumentsList[i]` <= 10
- It is guaranteed that the list of functions generates a valid composition.

Input Format For Custom Testing

The first line contains an integer, n, denoting the number of elements in `functionsList`.
Each line i of the n subsequent lines (where 0 <= i < n) contains a string describing `functionsList[i]`.
The first line contains an integer, q, denoting the number of elements in `argumentsList`.
Each line i of the q subsequent lines (where 0 <= i < q) contains an integer describing `argumentsList[i]`.

Sample Case 0

Sample Input For Custom Testing

```
STDIN       Function
-----       --------
3           functionList[] size n = 3
add         functionList = ["add", "splitter", "my_max"]
splitter
my_max
3           argumentsList[] size q = 3
2           argumentsList = [2, 1, 3]
1
3
```

Sample Output

```
3
```

Explanation

`compose(2, 1, 3)` is processed as shown below -

- `add(2, 1, 3)` -> 6
- `splitter(6)` -> [3, 3]
- `my_max([3, 3])` -> 3

Sample Case 1

Sample Input For Custom Testing

```
STDIN       Function
-----       --------
3           functionList[] size n = 3
add         functionList = ["add", "square", "splitter"]
square
splitter
2           argumentsList[] size q = 2
2           argumentsList = [2, 4]
4
```

Sample Output

```
[18, 18]
```

Explanation

`compose(2, 4)` is processed as shown below -

- `add(2, 4)` -> 6
- `square(6)` -> 36
- `splitter(36)` -> [18, 18]

*